ALEXANDRE Ewan, Groupe 1
18/10/2026

TD 2 : PIERRE FEUILLE PAPIER CISEAUX

Table des matières

1) Problématique 1
2) Exercices 1
Exercice 1 : Organisation du code et structure initiale 1
Exercice 2 : Écriture des premiers tests unitaires avec PHPUnit 2
Exercice 3 : Première implémentation de la fonction play 3
Exercice 4 : Refactoring avec des constantes de classe 4
Exercice 5 : Test et implémentation de la méthode party 4
Exercice 6 : Intégration et appel de la classe 6
3) Conclusion 7

1) Problématique

Nous cherchons à concevoir une classe de jeu Pierre Feuille Ciseaux en appliquant la démarche TDD (Test Driven Development) avec PHPUnit. Pour cela, nous allons écrire nos tests unitaires avant le code métier afin de définir précisément les comportements attendus, vérifier leurs échecs initiaux, puis coder la solution minimale et la refactoriser en toute sécurité.

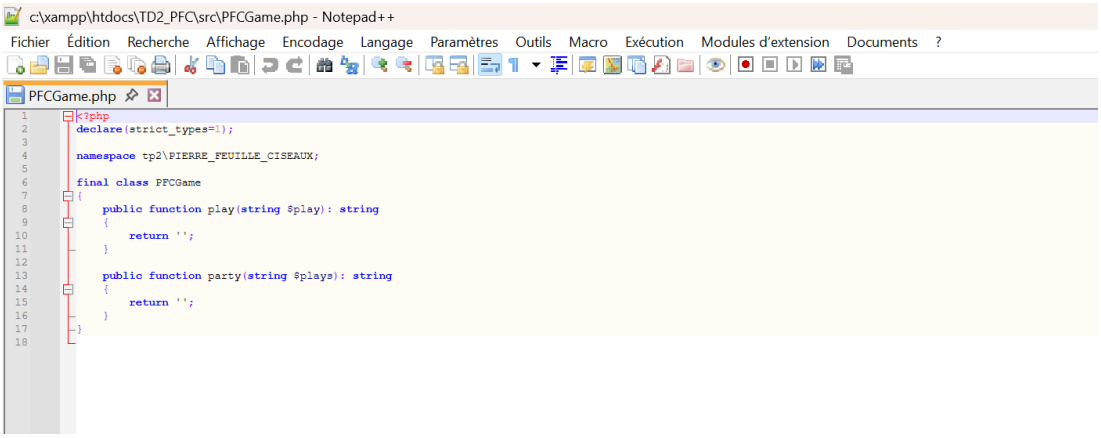
2) Exercices

Exercice 1 : Organisation du code et structure initiale

Pour réaliser ce TP, nous organisons notre projet avec deux classes principales :
- La classe métier PFCGame dans le fichier src/PFCGame.php, qui contiendra la logique du jeu.
- La classe de tests Pierre_feuille_ciseauxTest dans tests/Pierre_feuille_ciseauxTest.php pour exécuter PHPUnit.

Pour battre l'adversaire à chaque manche, nous prévoyons deux méthodes dans PFCGame :
- La méthode play(string $play): string retourne le coup gagnant pour un coup adverse unique.
- La méthode party(string $plays): string prend une séquence complète de coups et renvoie la suite des réponses gagnantes.

On commence par déclarer la classe PFCGame dans Notepad++ avec des retours vides return '' :

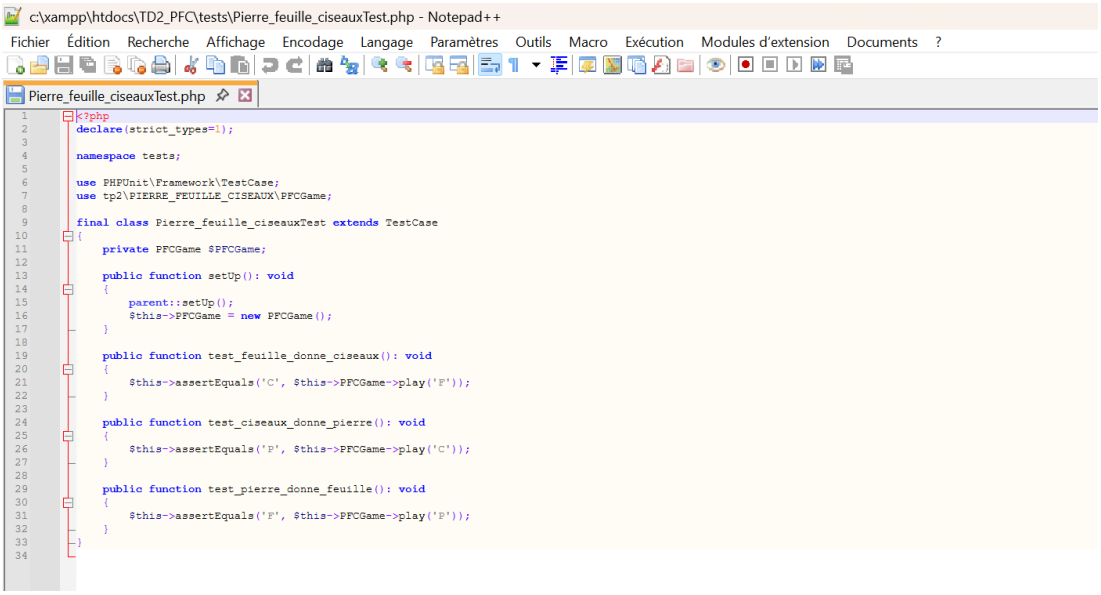


La structure initiale est créée sans code métier, prête pour l'écriture des premiers tests.

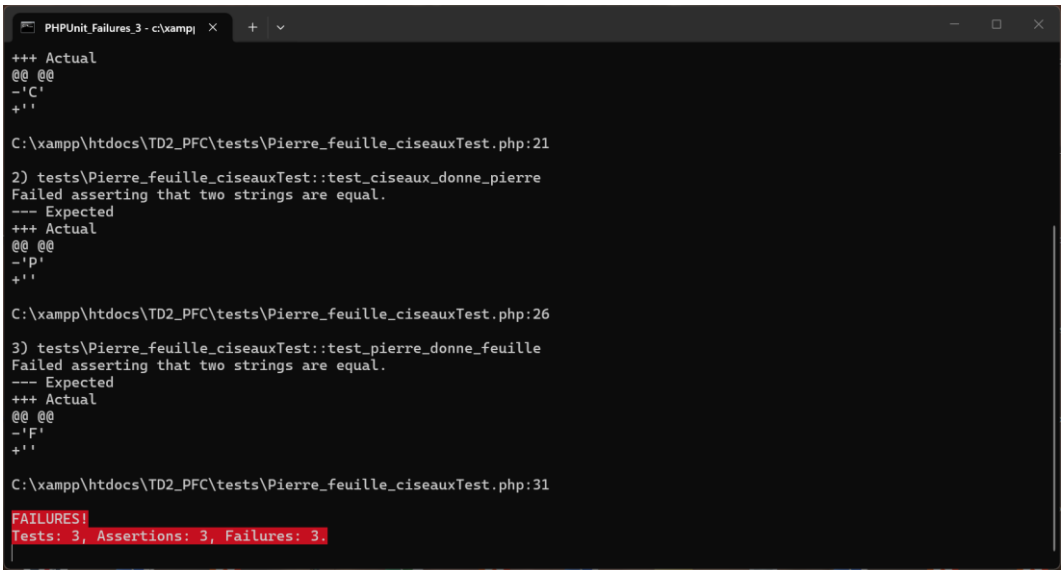
Exercice 2 : Écriture des premiers tests unitaires avec PHPUnit

En méthodologie TDD, on rédige les tests avant d'implémenter le fonctionnement réel de la méthode. Pour tester la méthode play, nous appliquons les règles classiques du jeu :
- Si l'adversaire joue 'F' (Feuille), on doit répondre 'C' (Ciseaux).
- Si l'adversaire joue 'C' (Ciseaux), on doit répondre 'P' (Pierre).
- Si l'adversaire joue 'P' (Pierre), on doit répondre 'F' (Feuille).

Dans Pierre_feuille_ciseauxTest.php, on instancie PFCGame dans la méthode setUp(), puis on écrit les 3 assertions avec assertEquals :

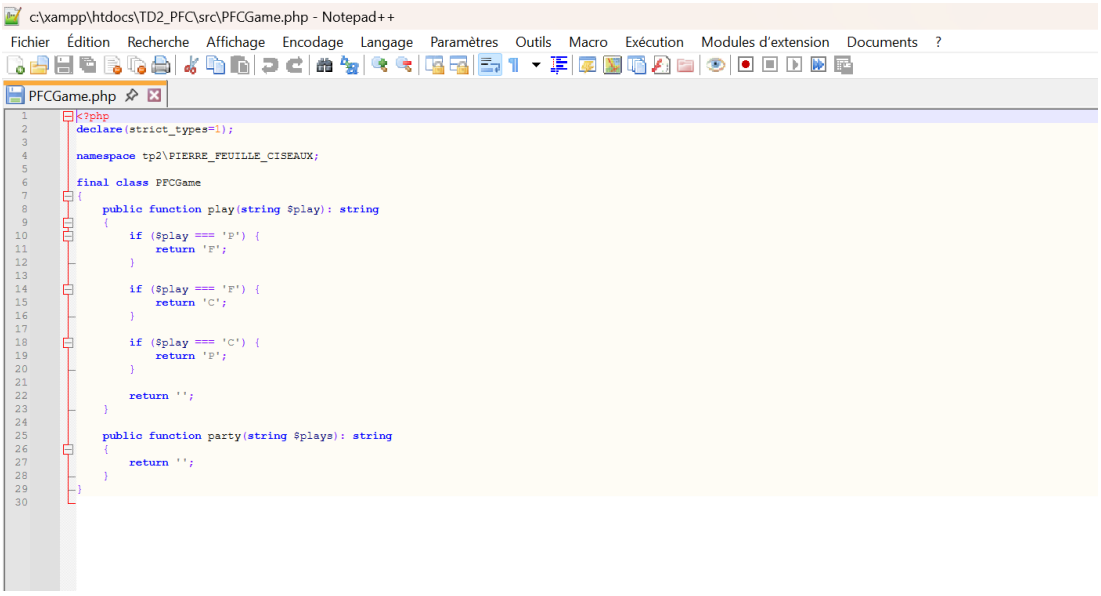


On lance ensuite l'exécution de PHPUnit dans le terminal Windows :

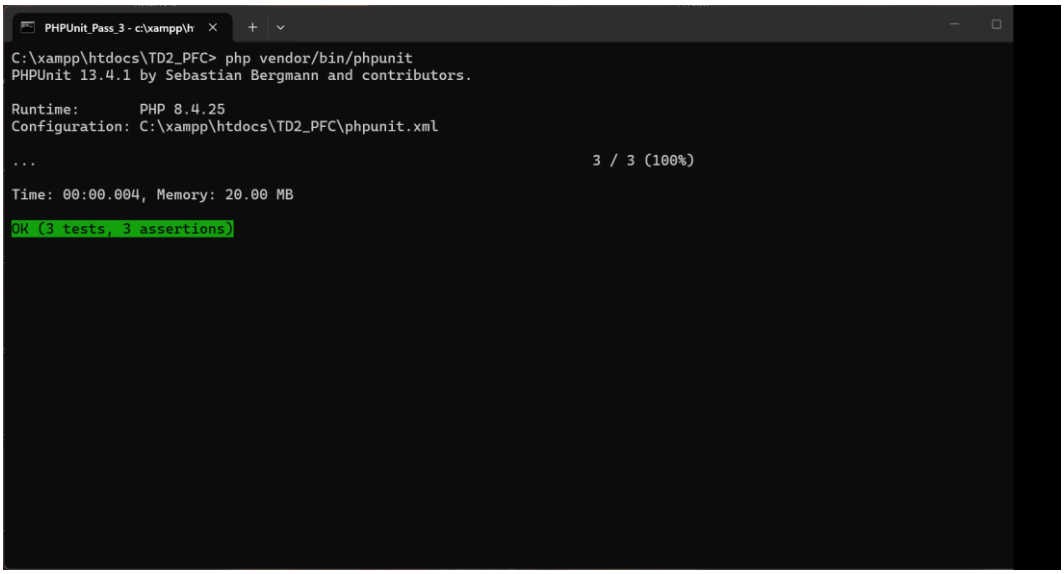


Exercice 3 : Première implémentation de la fonction play

Pour faire passer les tests au vert, on code la version la plus simple possible dans PFCGame.php en utilisant trois conditions if successives :



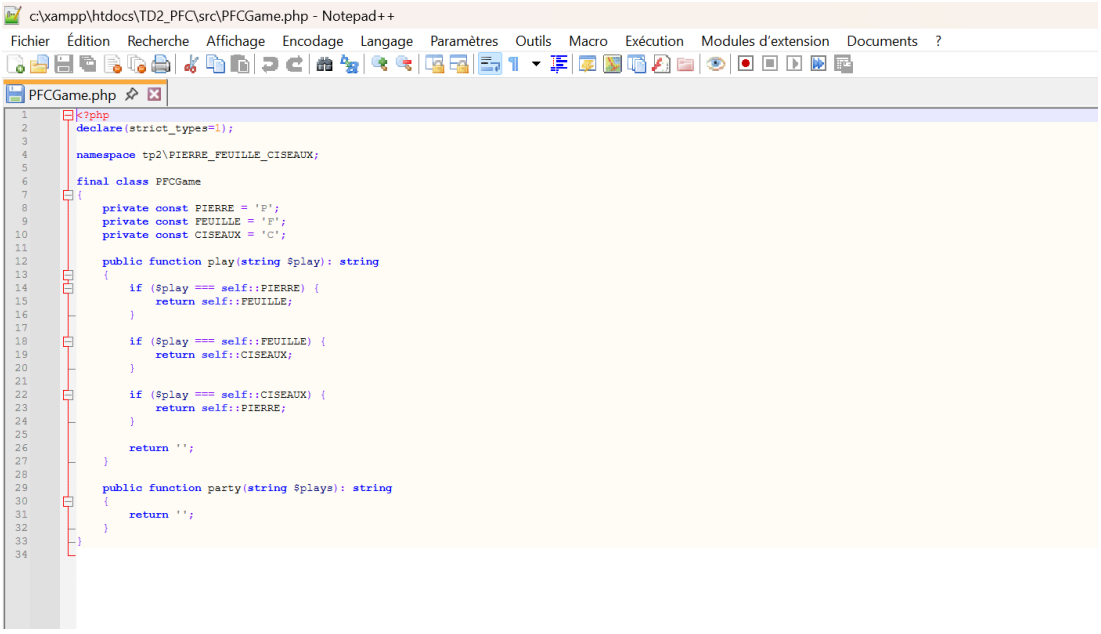
On relance immédiatement PHPUnit dans le terminal Windows :



Exercice 4 : Refactoring avec des constantes de classe

Maintenant que nos tests sont au vert, nous pouvons refactoriser notre code sans crainte de régression. On remplace les chaînes en dur 'P', 'F' et 'C' par des constantes de classe privées (PIERRE, FEUILLE, CISEAUX). Cela sécurise le typage et évite les erreurs de frappe : une mauvaise constante déclenche immédiatement une erreur PHP.

Voilà à quoi ressemble le code refactorisé dans Notepad++ :

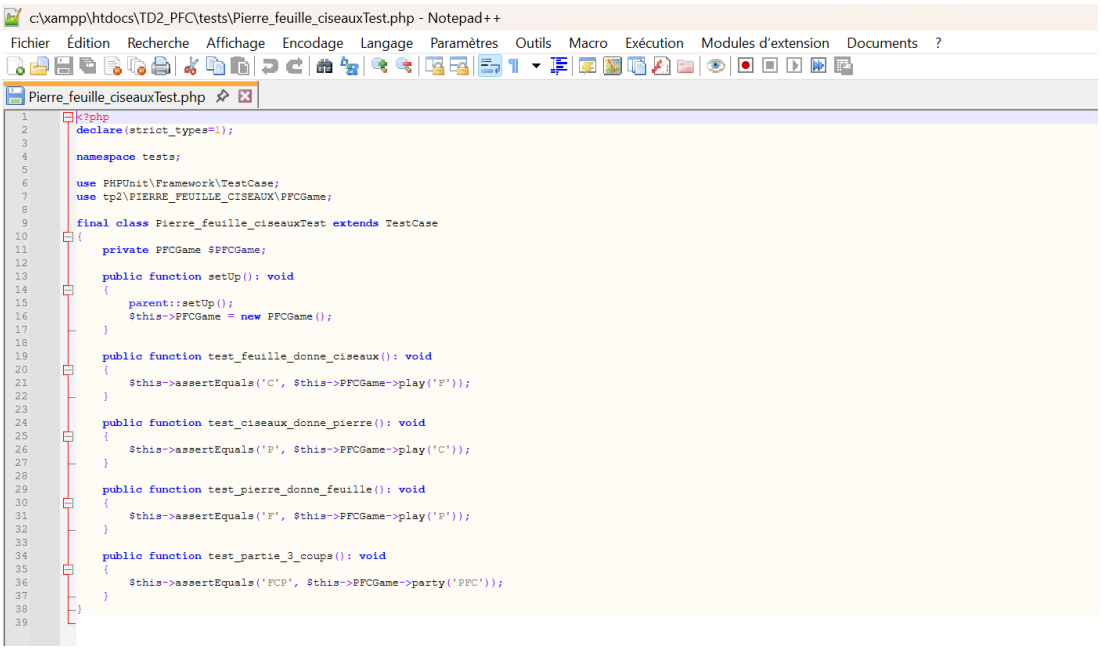


Grâce au filet de sécurité offert par nos tests automatisés, nous avons l'assurance que le comportement reste intact.

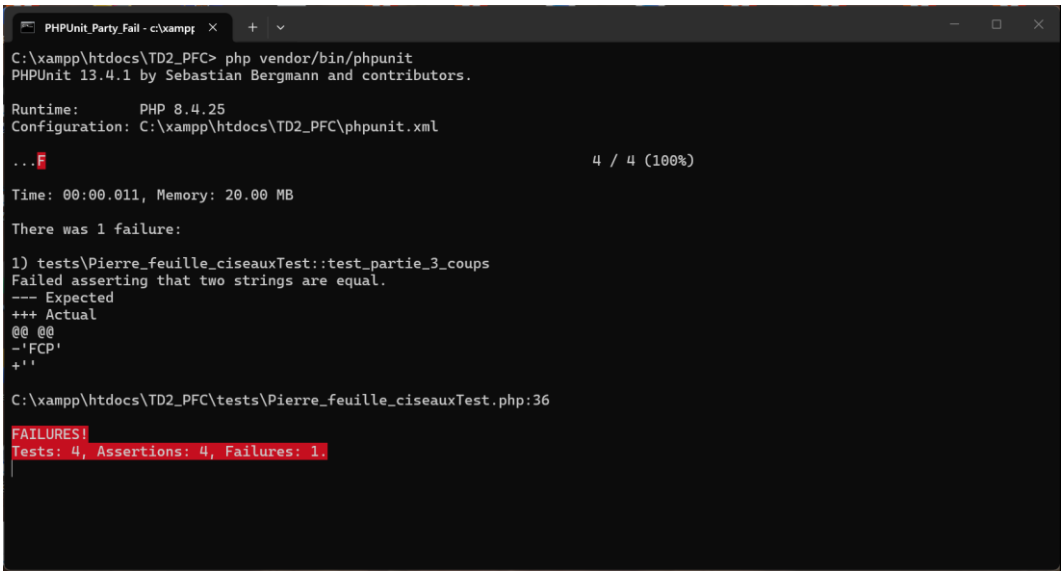
Exercice 5 : Test et implémentation de la méthode party

Nous devons maintenant tester et développer la méthode party. Celle-ci reçoit une chaîne de plusieurs coups adverses et doit retourner la chaîne correspondante des coups gagnants (par exemple 'PFC' doit donner 'FCP').

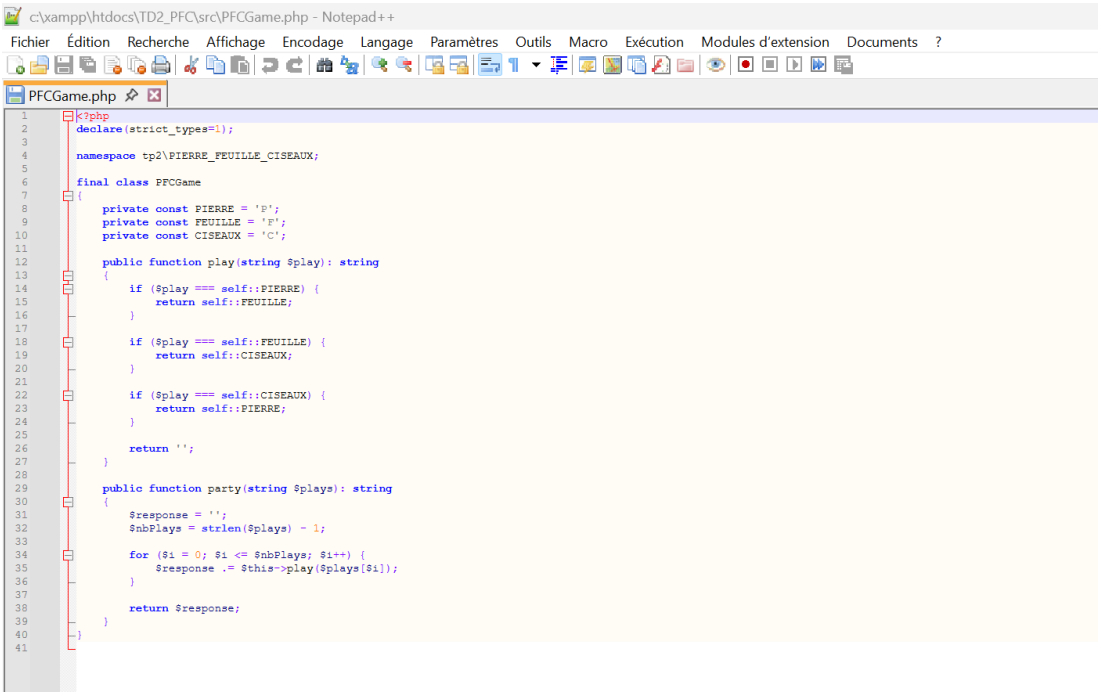
On commence par ajouter le test test_partie_3_coups dans notre suite de tests sous Notepad++ :



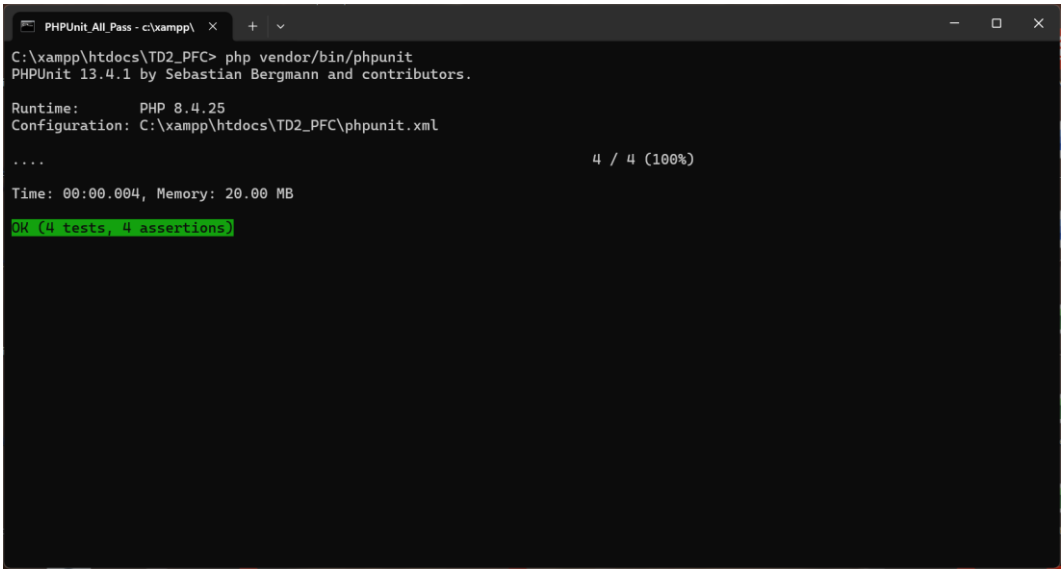
On lance les tests dans le terminal pour constater le nouvel échec :



On implémente ensuite la méthode party dans PFCGame.php en itérant sur la chaîne avec une boucle for et en réutilisant notre méthode play :

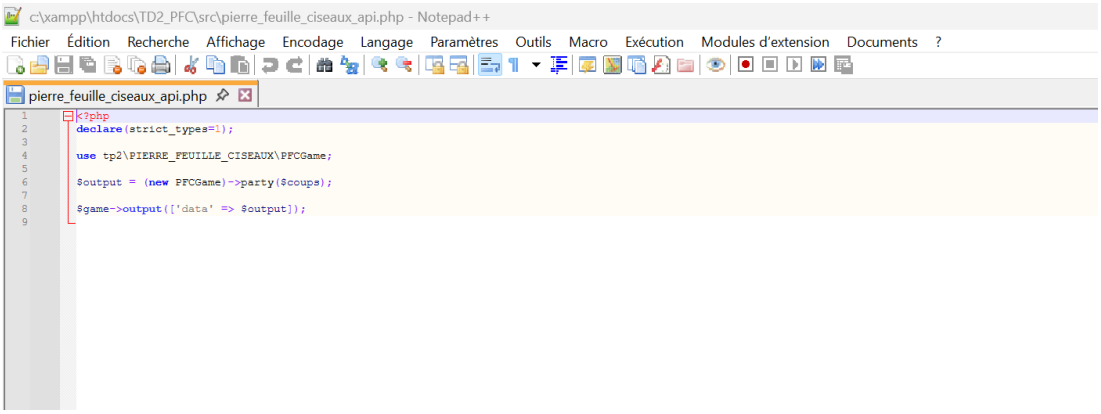


On relance une dernière fois les tests dans le terminal Windows :



Exercice 6 : Intégration et appel de la classe

Pour intégrer notre classe de jeu, on importe PFCGame dans le script pierre_feuille_ciseaux_api.php. On instancie la classe et on transmet la variable $coups à la méthode party, puis on transmet la réponse :



La méthode play n'étant appelée qu'en interne par party, elle pourra également être passée en private dans une démarche d'encapsulation avancée.

3) Conclusion

En conclusion, ce TP m'a permis d'assimiler les principes fondamentaux du Test Driven Development (TDD) avec PHPUnit. J'ai pu constater concrètement l'intérêt d'écrire les tests unitaires avant le code : cela force à concevoir clairement les interfaces, valide chaque étape au fur et à mesure et permet de refactoriser le code avec des constantes de classe en toute sérénité.